

SESIÓN 04

Enseñarle su dominio

Hugging Face, fine-tuning y LoRA — hoy su modelo aprende de sus datos

1

modelo base → especializado

3

librerías nuevas de HF

M1

Nos queda casi lista hoy

De modelo genérico a modelo suyo.

- 00 Sus dudas de la sesión pasada
- 01 El ecosistema Hugging Face
- 02 Fine-tuning con Trainer API
- 03 LoRA y QLoRA: adaptar sin pagar de más
- 04 Lab: fine-tuning de su modelo
- 05 Cierre y entrega M1

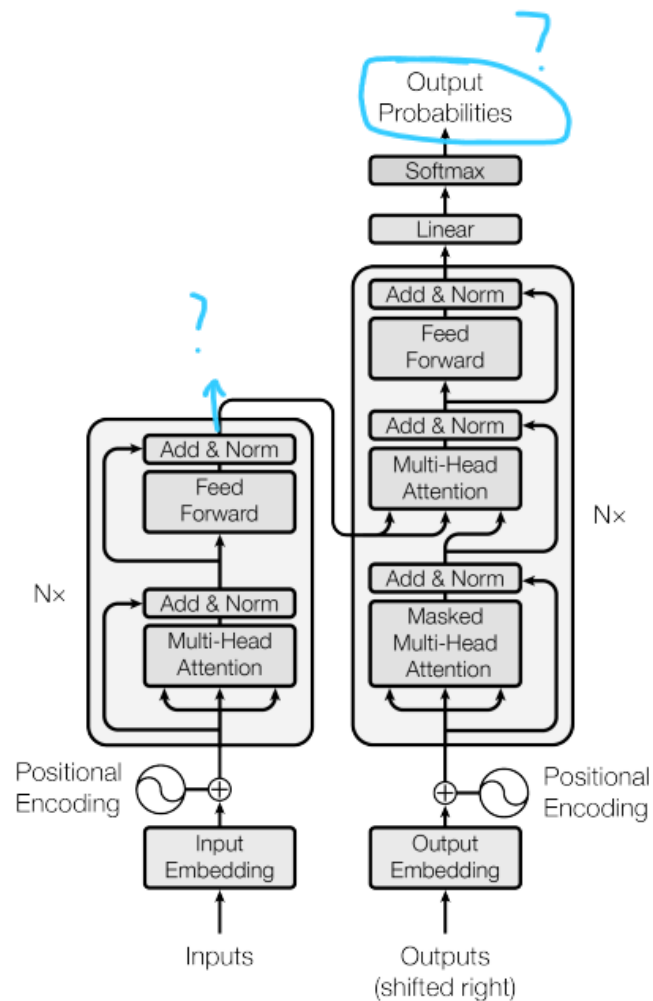
BLOQUE 00

Sus dudas

15 minutos · respuestas del formulario de salida de la S03

Qué sale de un encoder y de un decoder

¿los vectores son probabilidades, o tokens?



Un vector no es una probabilidad

La misma palabra «banco», saliendo de las dos formas. Miren qué es cada salida.

frase de entrada
«me senté en el banco»

Encoder (BERT)

saca un VECTOR: el significado de «banco» en contexto

banco →

0.82 -0.31 0.55 -0.12 0.94 0.07 -0.68 ...

Una lista de ~768 números. Cada uno es una coordenada del «significado». Juntos ubican la palabra en un espacio.

NO suman nada. NO son probabilidades.

Pueden ser negativos, grandes, cualquier valor. No dicen «qué palabra sigue» — dicen «qué significa esta palabra aquí». Sirven para clasificar o buscar.

Decoder (GPT, Qwen)

saca PROBABILIDADES: candidatos a la siguiente palabra

banco	64.8%
sofá	21.6%
piso	7.2%
parque	4.4%
sol	2.2%
suma	100% ✓

Sí suman 1. Sí son probabilidades.

Una por cada palabra del vocabulario. La más alta se elige como el siguiente token. Por eso «genera».

De hecho el decoder primero calcula un vector (igual que el encoder) y AL FINAL lo convierte en probabilidades.

Qué sale de un encoder y de un decoder

¿los vectores son probabilidades, o tokens?

Encoder (BERT)

Saca un VECTOR por cada palabra: su significado en contexto.
No son probabilidades ni tokens — son representaciones ricas.

Sirven para: clasificar, buscar por similitud, extraer. Se conectan a una capa extra que decide.

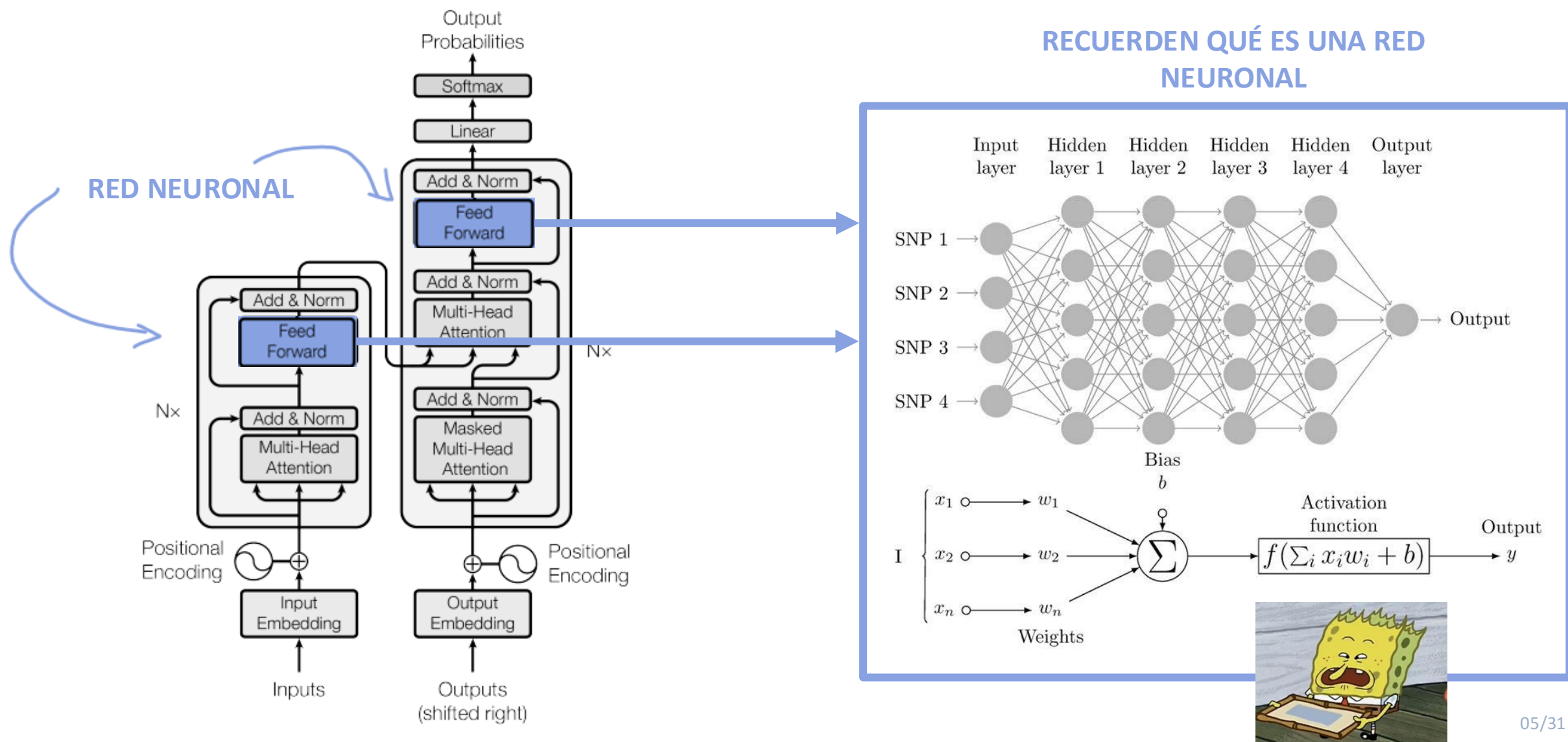
Decoder (GPT, Qwen)

Su capa final Sí produce probabilidades: una por cada palabra posible del vocabulario. La más probable se convierte en el siguiente token.

Por eso «genera»: elige token, lo agrega, y repite.
Vector → probabilidades → token.

Y sí: para un agente conversacional al que le das contexto, la opción es el decoder. Confirmado.

La FFN por dentro: perceptrón y no-linealidad



La FFN por dentro: perceptrón y no-linealidad

¿perceptrón, no linealidad?

Perceptrón

Una neurona: multiplica las entradas por pesos, las suma, y pasa el resultado por una función. La pieza más básica.

MLP / FFN

Varias capas de perceptrones, una tras otra. En el transformer: expande a 4× el tamaño, aplica la no-linealidad, y comprime de vuelta.

No-linealidad

La función que se aplica en medio (ReLU, GELU). Sin ella, apilar capas no serviría: dos capas lineales seguidas equivalen a una sola.

La no-linealidad es la razón de ser del MLP: es lo que le permite aprender patrones que una sola multiplicación no puede.

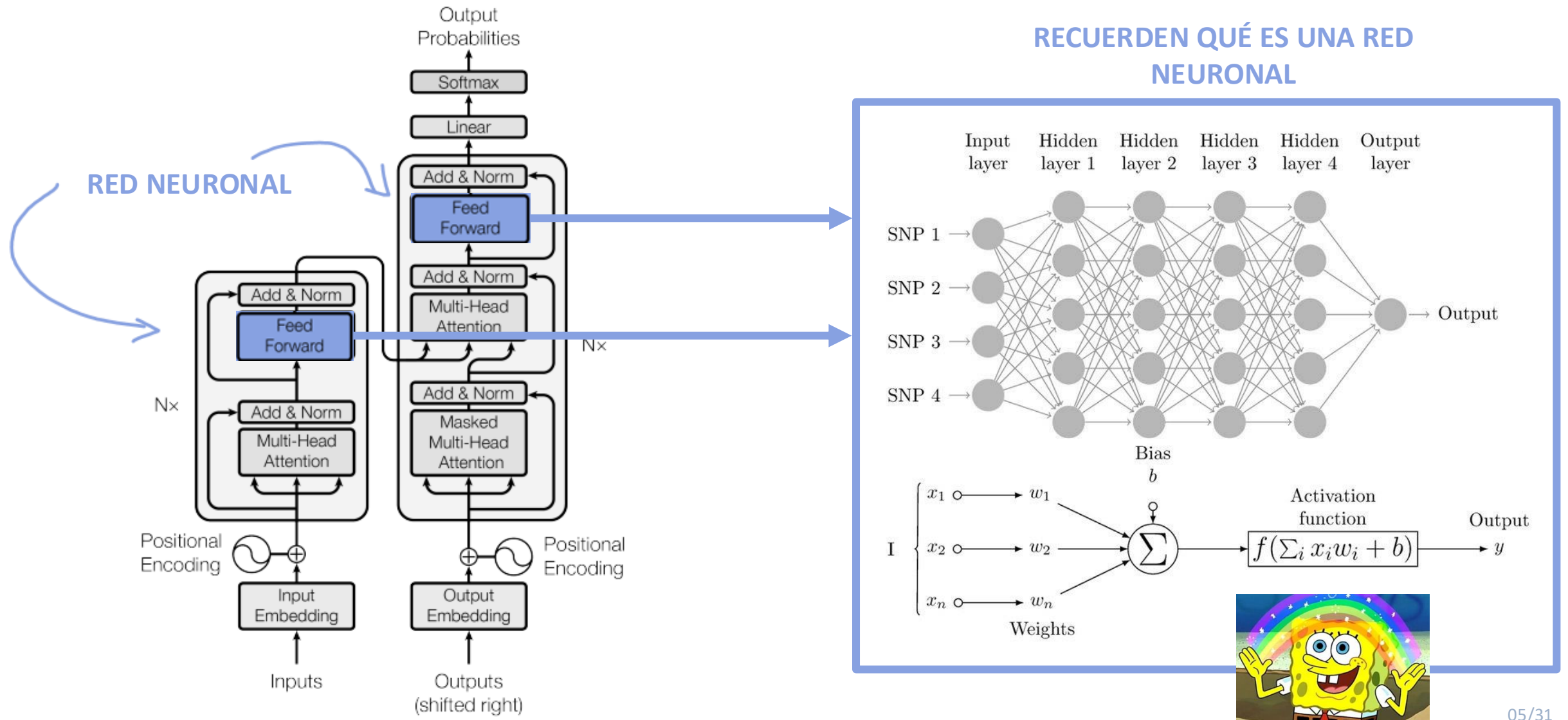
1 · la pieza mínima



Un **perceptrón**: pondera las entradas, las suma, y pasa el total por una **función**.

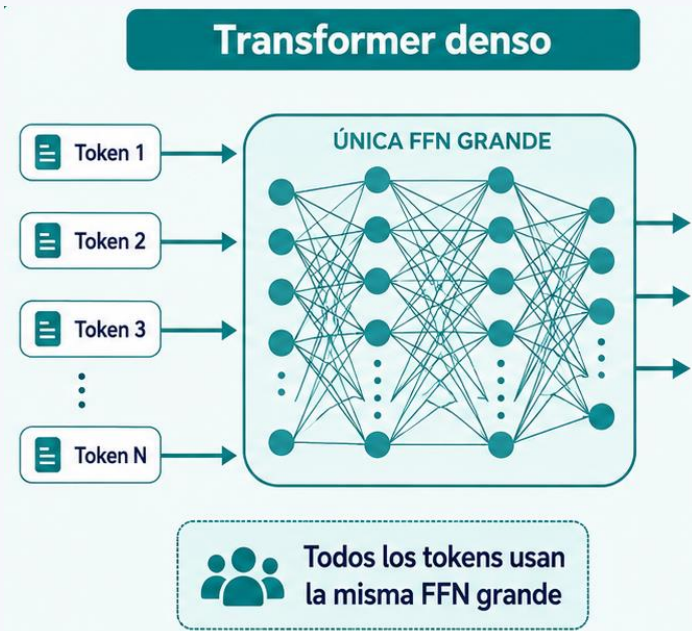
Perceptrón → MLP → por qué la función de activación es indispensable.

La FFN por dentro: perceptrón y no-linealidad

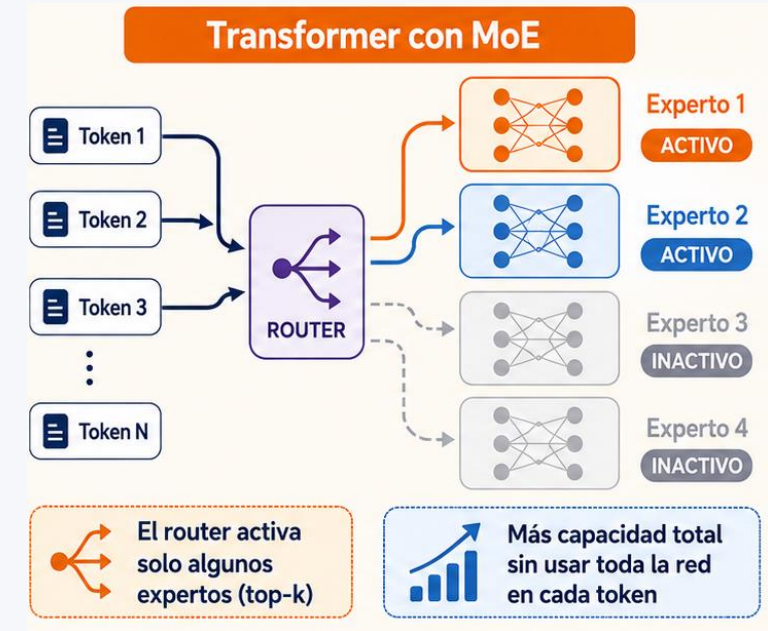


Mixture of Experts, sin misterio

En la FFN de cada bloque, en vez de UNA red grande, se ponen varias pequeñas (los «expertos»).



V.5.



Mixture of Experts, sin misterio

¿Cómo elige el router?

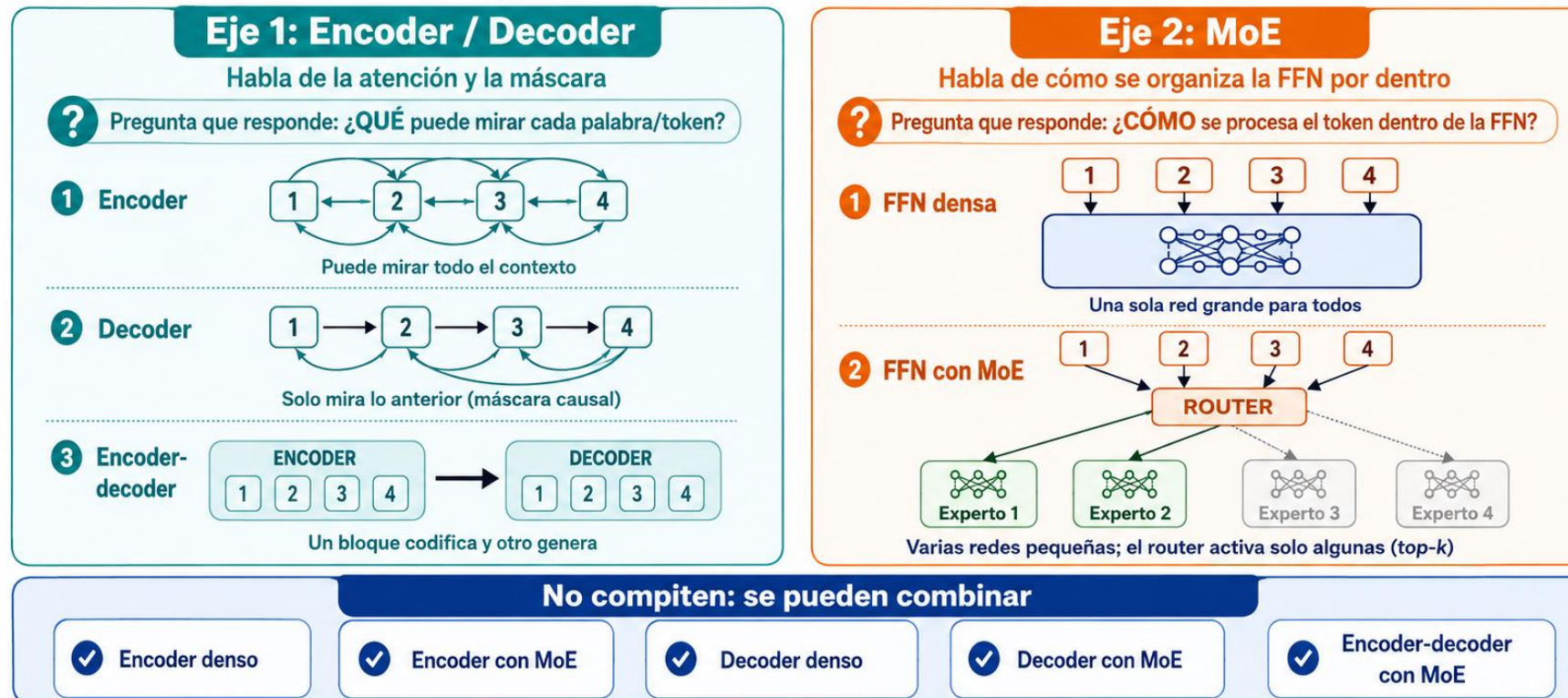
Es una capa pequeña que mira el token y da un puntaje a cada experto. Se quedan los de mayor puntaje. Se entrena junto con todo lo demás: nadie le dice a mano qué experto sirve para qué.



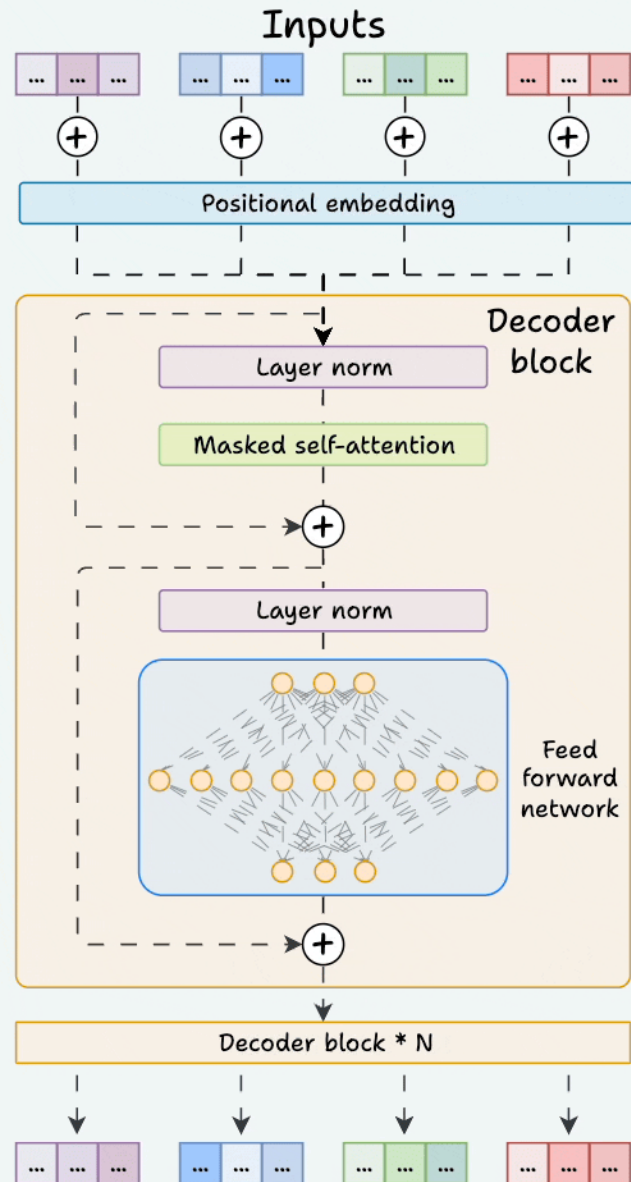
Mixture of Experts, sin misterio

¿Por qué NO es un encoder-decoder?

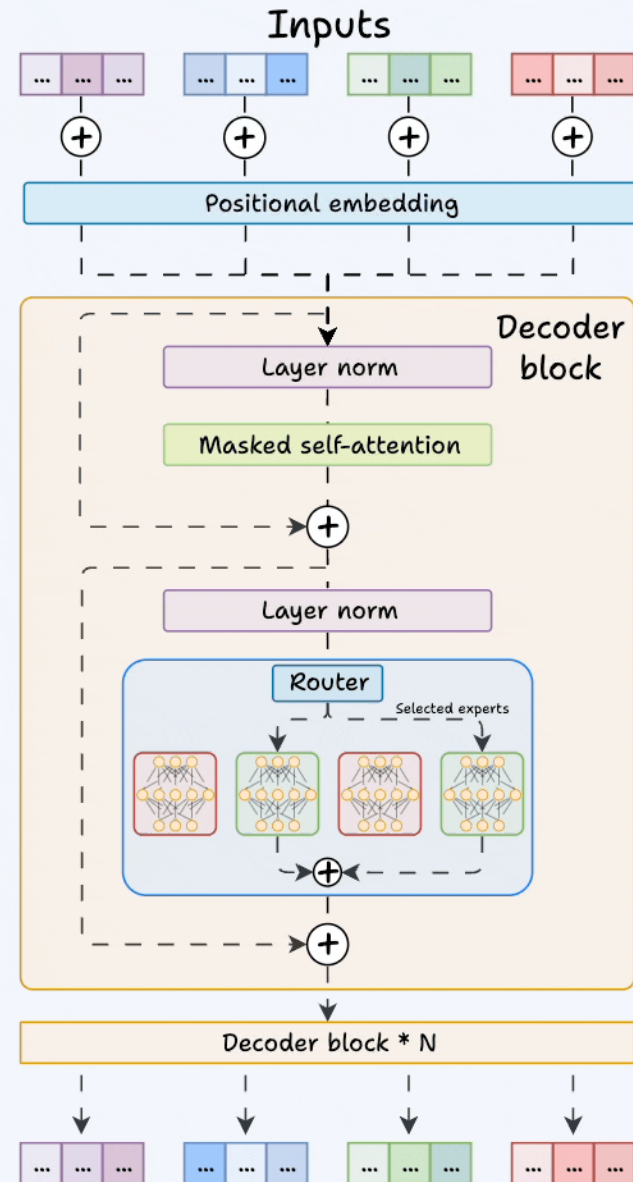
Son ejes distintos. Encoder/decoder es sobre QUÉ puede mirar cada palabra (la máscara). MoE es sobre CÓMO se organiza la FFN por dentro. Un decoder puede usar MoE, un encoder también. No compiten.



Transformer



Mixture of Experts



Span corruption y el <extra_id>

Original: El gato duerme en el sofá

Entrada: El gato <extra_id_0> el sofá (se tapa un TRAMO)

Salida: <extra_id_0> duerme en (reconstruye lo tapado)

El <extra_id_0> es solo una etiqueta: un marcador que dice «aquí faltaba algo». No es una palabra real, es el hueco numerado.

Span corruption: qué es y cuándo ocurre

No es código de PyTorch. Es un juego de entrenamiento — y solo pasa una vez en la vida del modelo.

¿Dónde ocurre? Solo en la fábrica, no cuando tú lo usas

FASE 1 · Pre-entrenamiento

Lo hacen Google/Meta una sola vez. Meses, millones de USD.

Aquí se usa span corruption: se tapan tramos y el modelo aprende a reconstruirlos. Es un JUEGO.



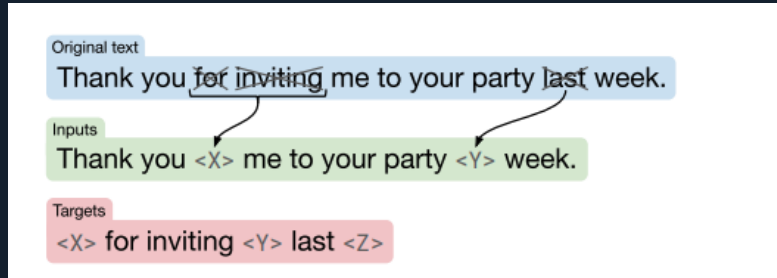
FASE 2 · Tú usas el modelo

El modelo ya está entrenado. Lo bajas del Hub y lo usas.

Ya **NO** hay span corruption. Le das texto, responde. El juego ya cumplió su función.

Como los ejercicios de práctica con que alguien aprendió: cuando ya sabe, no los repite — aplica lo aprendido.

Técnica



Pre entrenar un modelo
(span corruption a escala)



Texto entra -> Texto sale



Las dos fases, en código

Fíjate: span corruption está en la fábrica (izquierda). Cuando TÚ usas el modelo (derecha), no aparece por ningún lado.

FASE 1 · La fábrica (Google)

Aquí SÍ se programa span corruption. Una sola vez, con todo internet.

```
pre_entrenamiento.py

# se tapan tramos del texto al azar
texto = "El gato duerme en el sofá"
entrada= "El gato <X> el sofá"
target = "<X> duerme en"

# el modelo aprende a reconstruir
loss = model(entrada, target)
loss.backward()
```

Esto lo corre Google durante meses.

El resultado es el modelo que se sube al Hub.

google-t5/t5-base
modelo ya entrenado · listo en el Hub

*Tú nunca escribes este código.
Ya está hecho cuando llegas.*

bajas
→
el modelo

FASE 2 · Tú usas el modelo

Le das texto, te responde. Ni <extra_id>, ni tapar, ni target. Nada.

```
mi_app.py

from transformers import pipeline

# cargar el modelo ya entrenado
t5 = pipeline("summarization",
              "google-t5/t5-base")

# darle texto y recibir respuesta
texto = "summarize: El informe...
        trimestral muestra..."
salida = t5(texto)

print(salida)

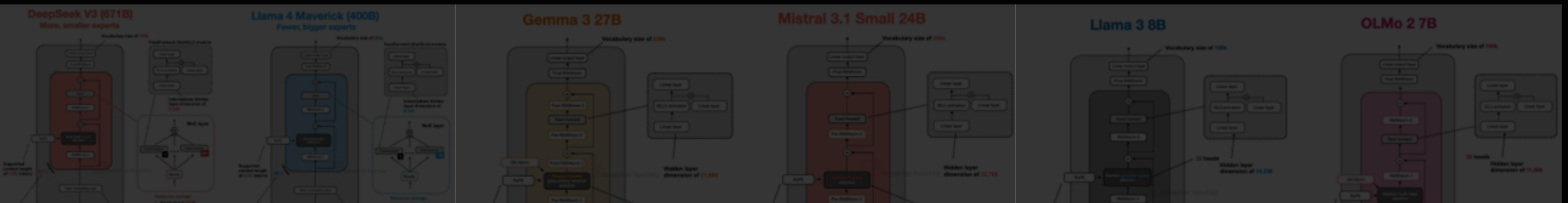
→ "Las ventas subieron 12% en
   el trimestre."
```

Texto entra → texto sale. Así de simple.

El span corruption ya cumplió su trabajo en la Fase 1.
Le enseñó al modelo a reconstruir texto — y esa
habilidad es la que ahora usas para resumir.

Para los curiosos... ya pueden entenderlo

<https://magazine.sebastianraschka.com/p/the-big-llm-architecture-comparison>





The AI community building the future.

The platform where the machine learning community collaborates on models, datasets, and applications.

[Explore AI Apps](#)

or

[Browse 2M+ models](#)[Main](#) [Tasks](#) [Libraries](#) [Languages](#) [Licenses](#) [Other](#)Models 2,555,000 [Filter by name](#)

Tasks

[Text Generation](#) [Any-to-Any](#)
[Image-Text-to-Text](#) [Image-to-Text](#)
[Image-to-Image](#) [Text-to-Image](#)
[Text-to-Video](#) [Text-to-Speech](#) [+ 44](#)

Parameters

[< 1B](#) [4B](#) [12B](#) [32B](#) [128B](#) [> 500B](#)

Libraries

[PyTorch](#) [TensorFlow](#) [JAX](#)
[Transformers](#) [Diffusers](#) [Safetensors](#)
[ONNX](#) [GGUF](#) [Transformers.js](#) [MLX](#)

Apps

[vLLM](#) [TGI](#) [llama.cpp](#) [MLX LM](#)
[LM Studio](#) [Ollama](#) [Jan](#) [+ 7](#)

Inference Providers

[Groq](#) [Novita](#) [Cerebras](#) [SambaNova](#)
[Nscale](#) [fal](#) [Hyperbolic](#) [Together AI](#)[moonshotai/Kimi-K2.5](#)[Image-Text-to-Text](#) • Updated 2 days ago • [↓ 96.2k](#) • [↑ 1.42k](#)[openai/gpt-oss-120b](#)[Text Generation](#) • [↓ 120B](#) • Updated Aug 26, 2025 • [↓ 2.88M](#) • [↑ 1.4k](#)[Lightricks/LTX-2](#)[Image-to-Video](#) • Updated 14 days ago • [↓ 2.72M](#) • [↑ 1.41k](#)[microsoft/VoiceVox-ASR](#)[Automatic Speech Recognition](#) • [↓ 9B](#) • Updated 6 days ago • [↓ 175k](#) • [↑ 1.4k](#)[nvidia/personalex-7b-v1](#)[Audio-to-Audio](#) • Updated 4 days ago • [↓ 101k](#) • [↑ 1.58k](#)[MiniMaxAI/MiniMax-M2.1](#)[Text Generation](#) • Updated 5 days ago • [↓ 84k](#) • [↑ 1.24k](#)[Qwen/Qwen3-ASR-0.6B](#)[Automatic Speech Recognition](#) • [↓ 0.6B](#) • Updated 3 days ago • [↓ 11.9k](#) • [↑ 128](#)[deepseek-ai/DeepSeek-OCR-2](#)[Text Generation](#) • Updated 4 days ago • [↓ 18.3k](#) • [↑ 621](#)[arcee-ai/Trinity-Large-Preview](#)[Text Generation](#) • [↓ 398B](#) • Updated 5 days ago • [↓ 317](#) • [↑ 115](#)[stepfun-ai/Step-3.5-Flash](#)[Text Generation](#) • Updated about 1 hour ago • [↓ 44](#) • [↑ 114](#)

BLOQUE 01

El ecosistema Hugging Face

las herramientas con las que van a trabajar

Hugging Face no es un modelo. Es un ecosistema.

Piénsenlo como el GitHub del machine learning: un lugar donde vive todo lo que necesitan.

transformers

Cargar y usar cualquier modelo. Ya lo conocen de S02–S03.

datasets

Cargar, transformar y compartir datasets sin llenar la RAM.

peft

Fine-tuning eficiente: LoRA y QLoRA viven aquí.

evaluate

Métricas estándar listas para medir su modelo.

accelerate

Corre el mismo código en CPU, una GPU o varias, sin cambiarlo.

Hub

El repositorio: modelos, datasets y demos, versionados como en Git.

El Hub, por dentro

Ya eligieron su modelo base la semana pasada. Aquí es donde vive, y donde van a leer su tarjeta.

Model card

La ficha del modelo: qué hace, con qué datos se entrenó, cómo usarlo. Es lo primero que leen.

Licencia

Qué pueden y no pueden hacer con él. Apache-2.0 y MIT son permisivas; otras restringen uso comercial.

Descargas y likes

Señal de si el modelo está vivo y mantenido. Un modelo abandonado los deja solos.

Files

Los pesos y la config. Lo que se descarga cuando llaman `from_pretrained`.

La tarea de la semana pasada era leer esta tarjeta. Quien la leyó, ya tiene medio M1 escrito.

Antes del modelo: los datos

Fine-tuning es enseñarle al modelo con ejemplos. Sin datos buenos, no hay nada que enseñar.

Un ejemplo de fine-tuning es un par: entrada → salida deseada

Educación: "explícame las fracciones" → una explicación clara para un niño de 10 años

Legal: "¿puedo terminar mi contrato?" → respuesta citando el artículo aplicable

● Cantidad

Con 20–200 ejemplos buenos ya se ve efecto usando LoRA. No necesitan millones.

● Calidad

10 ejemplos excelentes le enseñan más que 100 mediocres. La basura entra, la basura sale.

● Formato

datasets los carga desde CSV, JSON o el Hub. Hoy usan los 20 que trajeron.

Los 20 ejemplos que trajeron hoy son la semilla. En su proyecto real necesitarán más, pero hoy alcanzan para ver la magia.

Pausa

15 minutos

Al volver: cómo entrenar sin fundir la GPU gratis de Colab.

BLOQUE 02

Fine-tuning con Trainer

el entrenamiento clásico, de principio a fin

¿Qué es fine-tuning, en una imagen?

El modelo base ya sabe español y sabe del mundo. No parten de cero: parten de alguien que ya estudió.

Pre-entrenamiento

Alguien leyó media internet y aprendió el idioma y el mundo. Lo hicieron OpenAI, Meta, Alibaba — con millones de dólares en cómputo.

Fine-tuning ← ustedes

Toman a ese experto general y le enseñan SU tarea con unos pocos ejemplos. Como un posgrado corto. Cabe en Colab y en minutos.

La analogía

Contratar a alguien que ya habla español y sabe escribir, y darle una semana de inducción sobre SU empresa. No le enseñan a leer: le enseñan su contexto. Eso es fine-tuning.

Qué pasa cuando entrena

El mismo bucle que vieron en Redes Neuronales, ahora sobre su modelo.

1

Predice

El modelo intenta producir la salida a partir de la entrada

2

Compara

Mide qué tan lejos estuvo de la salida correcta: eso es la pérdida (loss)

3

Ajusta

Corrige sus números un poquito en la dirección que reduce el error

4

Repite

Miles de veces, con todos los ejemplos, varias vueltas (épocas)

La pérdida bajando es la señal de que aprende. La vamos a ver en vivo en el laboratorio.

Trainer: no reinventamos el bucle

Hugging Face ya escribió ese loop por ustedes. Se llama Trainer, y solo hay que configurarlo.

```
from transformers import Trainer, TrainingArguments

args = TrainingArguments(
    learning_rate=2e-4,
    num_train_epochs=3,
    per_device_train_batch_size=8,
    report_to="wandb",
)

trainer = Trainer(model, args, train_dataset)
trainer.train()
```

Cuatro líneas para entrenar. Trainer se encarga del bucle, el guardado, la evaluación y el reporte a W&B.

learning_rate	qué tan grande es cada paso de corrección	num_train_epochs	cuántas vueltas dan a todos los datos	batch_size	cuántos ejemplos ve antes de corregir
----------------------	---	-------------------------	---------------------------------------	-------------------	---------------------------------------

Ver el entrenamiento: **Weights & Biases**

Entrenar a ciegas es peligroso. W&B dibuja la pérdida en tiempo real para que vean qué pasa.

- **La pérdida baja**

El modelo aprende. Es lo que quieren ver: una curva que desciende.

- **La pérdida se estanca**

Ya aprendió lo que podía. Más épocas no ayudan — es hora de parar.

- **La pérdida sube u oscila**

Algo anda mal: learning rate muy alto, o datos ruidosos. Señal de alarma.

Para el proyecto: W&B no es opcional. La entrega final pide observabilidad verificable — y empieza hoy, con esta curva.

BLOQUE 03

LoRA y QLoRA

adaptar un modelo sin pagar por un modelo entero

El problema con el fine-tuning clásico

Lo que acabamos de ver funciona, pero tiene un costo que no cabe en Colab.

Fine-tuning completo = reentrenar TODOS los parámetros

Un modelo de 3 mil millones de parámetros necesita guardar en memoria no solo esos 3 mil millones, sino también sus gradientes y los estados del optimizador. En la práctica, entre 4 y 8 veces el tamaño del modelo. La T4 de Colab no aguanta.

La pregunta de LoRA

¿Y si en vez de reentrenar los 3 mil millones de parámetros, entrenamos solo un puñado pequeño que se le agrega al lado — dejando el modelo original congelado e intacto?

LoRA, en una idea

Low-Rank Adaptation. Congela el modelo original y le pega dos matrices pequeñas que sí se entrenan.

El modelo original



Congelado. Sus miles de millones de parámetros no se tocan. Se quedan exactamente como venían del Hub.

Las matrices LoRA

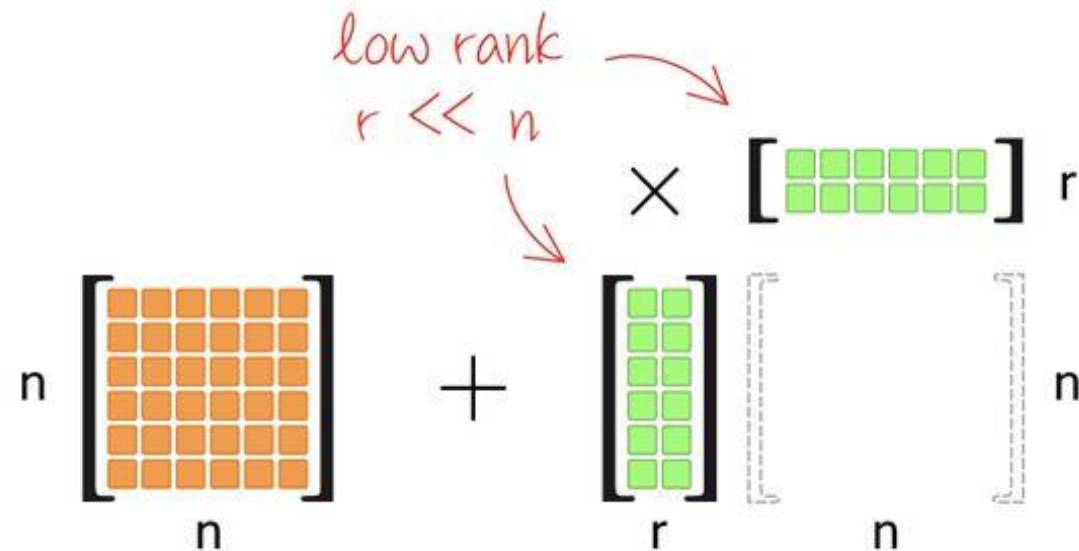


Dos matrices pequeñas, nuevas, que sí se entrenan. Son diminutas: menos del 1% del tamaño del modelo. Ahí queda guardado lo que aprendió de su dominio.

El resultado

Entrenan menos del 1% de los parámetros, cabe en Colab, entrena en minutos, y el archivo que guardan pesa megabytes en vez de gigabytes. Misma calidad para su tarea.

LoRA



$$W = W_0 + \underbrace{A}_{\text{trainable}} \underbrace{B}_{\text{trainable}}$$

Diagram illustrating the LoRA matrix decomposition. A large orange matrix of size $n \times n$ is added to the product of a low-rank matrix (green, size $n \times r$) and another matrix (green, size $r \times n$). Red arrows point from the text "low rank $r \ll n$ " to the green matrices.

Los tres números de LoRA

Cuando configuren LoRA verán estos tres. Esto es lo que significan, sin matemática.

rank (r)

El tamaño de las matrices nuevas. Más grande = más capacidad de aprender, pero más peso. Valores típicos: 8, 16, 32.

Empiecen en 8 o 16.

alpha

Cuánto pesa lo aprendido por LoRA frente al modelo original. Regla común: $\alpha = 2 \times \text{rank}$.

alpha = 16 o 32.

target_modules

A qué partes del modelo se les pega LoRA. Normalmente las capas de atención (q_proj, v_proj).

Dejen el default.

Para hoy: los defaults funcionan. No optimicen hiperparámetros antes de tener el pipeline corriendo.

QLoRA: LoRA + compresión

La Q es de quantized. Si LoRA no alcanza a caber, QLoRA comprime el modelo congelado para que quepa.

Idea: guardar el modelo congelado con menos precisión (4 bits en vez de 16). Ocupa la cuarta parte de memoria, y como está congelado, esa pérdida de precisión casi no afecta.

Cuándo LoRA

El modelo cabe cómodo en memoria. Más rápido, sin pérdida de precisión.

1B–3B en Colab

Cuándo QLoRA

El modelo es más grande y no cabe de otra forma. Un poco más lento, pero entra.

7B–8B en Colab

Regla para hoy: si su modelo es de 1B a 3B, usen LoRA. Si eligieron uno de 7B, QLoRA. El notebook trae las dos rutas.

BLOQUE 04

AI laboratorio

hoy su modelo aprende de sus datos

Lab: fine-tuning de su modelo

El notebook los lleva de la mano por el pipeline completo. Cada equipo con su modelo y sus 20 ejemplos.



Cargar y medir el baseline

Cargan su modelo base y miden qué tan bien hace la tarea SIN entrenar. Este número es su punto de comparación.



Fine-tuning con LoRA

Preparan sus datos, configuran LoRA, y entrenan. Ven la pérdida bajar en W&B, en vivo.



Medir la mejora

Vuelven a medir con el modelo entrenado y comparan contra el baseline. ¿Mejóro? ¿Cuánto?

El baseline del Lab A no es opcional: sin él no pueden demostrar que su fine-tuning sirvió. Es el corazón de M1.

AI notebook — Lab de fine-tuning

Cargar baseline · entrenar con LoRA · medir la mejora

```
baseline → LoRA → evaluación → S04_Lab_Fine_tuning.ipynb
```

Runtime → cambiar tipo de entorno → T4 GPU · antes de empezar

BLOQUE 05

Entrega M1

Entrega M1 — qué incluye

Entrega de equipo · 10% · sale casi completa del laboratorio de hoy · cierra el domingo 16 de agosto.

Modelo fine-tuneado

Su modelo base adaptado con LoRA sobre los datos del dominio. El notebook ejecutado.

Dataset documentado

Los ejemplos que usaron, de dónde salieron, y por qué son representativos de su tarea.

Baseline vs. resultado

El número de antes y el de después. La evidencia de que el fine-tuning sirvió.

Justificación del modelo

Por qué eligieron ESE modelo base: familia, idioma, tamaño. Lo que vienen construyendo desde S03.

Si el laboratorio les corrió, tienen los tres primeros hechos. El cuarto lo vienen escribiendo desde la sesión pasada.

Cómo se califica M1

Para que sepan dónde poner el esfuerzo. El detalle completo está en la rúbrica del repo.

QUÉ EVALÚA	QUÉ BUSCAMOS
El pipeline corre	El notebook entrena de principio a fin sin errores, con GPU.
Datos con sentido	Los ejemplos son reales, del dominio, y están documentados.
Mejora medible	El baseline y el resultado están, y se ve la diferencia.
Decisión justificada	La elección del modelo tiene argumentos, no 'salía en el tutorial'.

Lo que MÁS pesa no es que el número mejore mucho. Es que ustedes entiendan y puedan explicar por qué mejoró (o por qué no).

Tarea para S05

1

Terminar y subir M1

El notebook ejecutado, el dataset documentado y la comparación baseline vs. resultado, en el repo del equipo. Tienen hasta el domingo 16 de agosto.

2

Pensar cómo evaluar mejor

Hoy midieron con una métrica simple. En S05 empieza el módulo de evaluación: ¿cómo saben de verdad si su modelo es bueno?

S05 abre el Módulo 2: Evaluación. Dejan de preguntarse 'entrena?' y empiezan a preguntarse 'es bueno de verdad?'.

Salida de sesión

El formulario del canal. Dos minutos. Lo que menos entendieron hoy define con qué arranca la próxima clase.

Lo que se llevan

1

Fine-tuning es enseñar con ejemplos

No parten de cero: toman un experto general y le enseñan su dominio con pocos datos.

2

LoRA hace que quepa en Colab

Congela el modelo y entrena matrices diminutas. Menos del 1% de los parámetros.

3

Sin baseline no hay evidencia

Medir antes y después es lo que convierte 'entrené algo' en 'mejoró esto'.

Y la cuarta: hoy su proyecto pasó de idea a sistema. Tienen un modelo que hace algo que antes no hacía.

Su modelo ya sabe de lo suyo.

Módulo 1, completo. Construyeron, entendieron y entrenaron un transformer para su dominio. La próxima pregunta es más difícil: ¿es bueno de verdad?

S05 · Módulo 2 · Evaluación

Nos vemos el jueves.